2.3.2 Implementação de Filas Através de Apontadores

Assim como em todas as outras implementações deste capítulo, uma célula cabeça é mantida para facilitar a implementação das operações Enfileira e Desenfileira quando a fila está vazia, conforme ilustra a Figura 2.8. Quando a fila está vazia os apontadores Frente e trás apontam para a célula cabeça. Para enfileirar um novo item basta criar uma célula nova, ligá-la após a célula que contém x_n e colocar nela o novo item. Para desenfileirar o item x_1 basta desligar a célula cabeça da lista e a célula que contém x_1 passa a ser a célula cabeça.

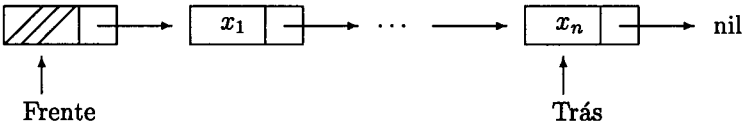

Figura 2.8: Implementação de uma fila através de apontadores

A fila é implementada através de células, onde cada célula contém um item da fila e um apontador para outra célula, conforme ilustra o Pro-grama 2.18. 0 registro TipoFila contém um apontador para a frente da fila (célula cabeça) e um apontador para a parte de trás da fila.

As quatro operações definidas sobre o TipoFila podem ser implementadas conforme ilustra o programa 2.19.

Notas Bibliográficas

Knuth (1968) apresenta um bom tratamento sobre listas lineares. Outras referências relevantes sobre estruturas de dados básicas são Aho, Hoperoft e Ullman (1983), Cormem, Leiserson e Rivest (1990) e Sedgewick (1988).

Exercícios

1) Considere a implementação de listas lineares utilizando apontadores e com célula-cabeça. Considere que um dos campos do TipoItem é uma Chave: TipoChave. Escreva uma função em Pascal
function EstaNaLista (Ch: TipoChave; var L: TipoLista): boolean;
que retorna true se Ch estiver na lista e retorna false se Ch não estiver na lista. Considere que não há ocorrências de chaves repetidas na lista. Determine a complexidade do seu algoritmo.

```
type
   Apontador = ^Célula;
   TipoItem  = record
                 Chave : TipoChave;
                 {outros componentes}
               end;
   Célula    = record
                 Item : TipoItem;
                 Prox : Apontador;
               end;
   TipoFila  = record
                 Frente: Apontador;
                 Trás  : Apontador;
               end;
```

Programa 2.18: Estrutura da fila usando apontadores

2) Um problema que pode surgir na manipulação de listas lineares simples é o de "voltar" atrás na lista, ou seja, percorrê-la no sentido inverso ao dos apontadores. A solução geralmente adotada é a incorporação à célula de um apontador para o seu antecessor. Listas deste tipo são chamadas de duplamente encadeadas. A Figura 2.9 mostra uma lista deste tipo com estrutura circular e a presença de uma célula cabeça.

   a) Declare os tipos necessários para a manipulação da lista.
   b) Escreva um procedimento em Pascal para retirar da lista a célula apontada por p:
      procedure Retira (p: Apontador; **var L**: TipoLista);
      Não deixe de considerar eventuais casos especiais.

3) Matrizes Esparsas – Utilização de Listas Através de Apontadores (Árabe, 1992)

   Matrizes esparsas são matrizes nas quais a maioria das posições são preenchidas por zeros. Para estas matrizes, podemos economizar um espaço significativo de memória se apenas os termos diferentes de zero forem armazenados. As operações usuais sobre estas matrizes (somar, multiplicar, inverter, pivotar) também podem ser feitas em tempo muito menor se não armazenarmos as posições que contém zeros.

   Uma maneira eficiente de representar estruturas com tamanho variável e/ou desconhecido é através de alocação encadeada, utilizando listas. Vamos usar esta representação para armazenar as matrizes esparsas.

```
procedure FFVazia (var Fila : TipoFila);
begin
  new (Fila.Frente);
  Fila.Trás := Fila.Frente;
  Fila.Frente^.Prox := nil;
end;
function Vazia (var Fila : TipoFila) : boolean;
begin
  Vazia := Fila.Frente = Fila.Trás;
end;
procedure Enfileira (x : TipoItem; var Fila : TipoFila);
begin
  new (Fila.Trás^.Prox);
  Fila.Trás := Fila.Trás^.Prox;
  Fila.Trás^.Item := x;
  Fila.Trás^.Prox := nil;
end;
procedure Desenfileira(var Fila : TipoFila; var Item : TipoItem);
var q : Apontador;
begin
  if Vazia (Fila)
  then writeln (' Erro : fila está vazia')
  else begin
       q := Fila.Frente;
       Fila.Frente := Fila.Frente^.Prox;
       Item := Fila.Frente^.Item;
       dispose(q);
       end;
end;
```

Programa 2.19: Operações sobre filas usando apontadores

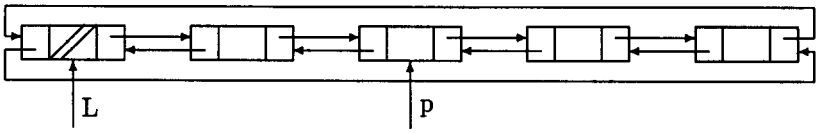

Figura 2.9: Lista circular duplamente encadeada

Cada coluna da matriz será representada por uma lista linear circular-Listas!circulares com uma célula cabeça. Da mesma maneira, cada linha da matriz também será representada por uma lista linear circular com uma célula cabeça. Cada célula da estrutura, além das células-cabeça, representará os termos diferentes de zero da matriz e devem ter o seguinte tipo:

```
type
    Apontador = ^ TipoCélula;

    TipoCélula = record
                     Direita,
                     Abaixo  : Apontador;
                     Linha,
                     Coluna  : integer;
                     Valor   : real;
                 end;
```

O campo Abaixo deve ser usado para apontar o próximo elemento diferente de zero na mesma coluna. O campo Direita deve ser usado para apontar o próximo elemento diferente de zero na mesma linha. Dada uma matriz A, para um valor $A(i,j)$ diferente de zero, deverá haver uma célula com o campo Valor contendo $A(i,j)$, o campo Linha contendo i e o campo Coluna contendo j. Esta célula deverá pertencer à lista circular da linha i e também deverá pertencer à lista circular da coluna j. Ou seja, cada célula pertencerá a duas listas ao mesmo tempo. Para diferenciar as células cabeça, coloque -1 nos campos Linha e Coluna destas células.

Considere a matriz esparsa seguinte

$$A = \begin{pmatrix} 50 & 0 & 0 & 0 \\ 10 & 0 & 20 & 0 \\ 0 & 0 & 0 & 0 \\ -30 & 0 & -60 & 5 \end{pmatrix}$$

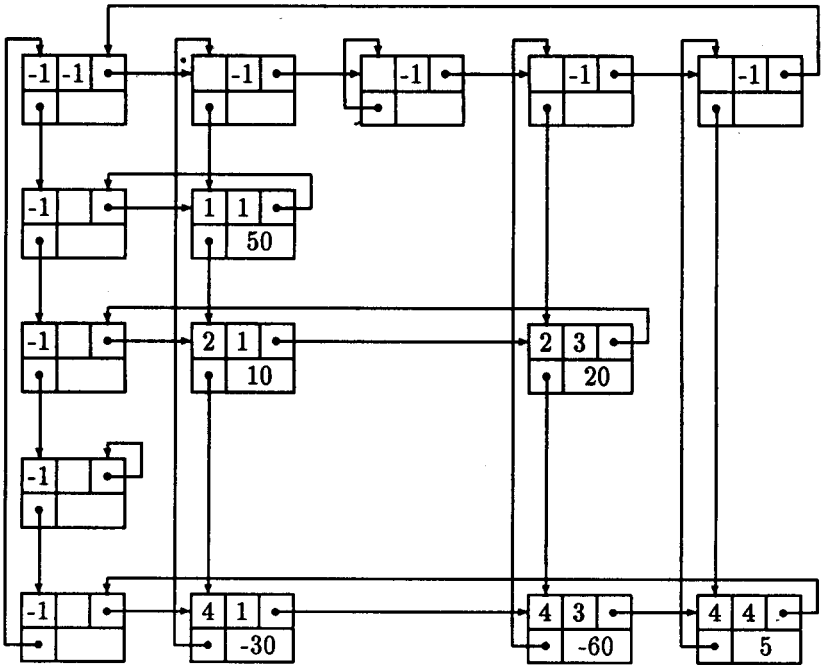

Figura 2.10: Exemplo de Matriz Esparsa

A representação da matriz A pode 'ser vista na Figura 2.10.

Com esta representação, uma matrix esparsa $m \ x \ n$ com r elementos diferentes de zero gastará $(m+n+r)$ células. E bem verdade que cada célula ocupa vários bytes na memória; no entanto, o total de memória usado será menor do que as $m \ x \ n$ posições necessárias para representar a matriz toda, desde que r seja suficientemente pequeno.

Dada a representação acima, o trabalho consiste em desenvolver cinco procedimentos em Pascal, conforme especificação abaixo:

a) procedure LeMatriz (var A: Matriz);

Este procedimento lê de algum arquivo de entrada os elementos diferentes de zero de uma matriz e monta a estrutura especificada acima. Considere que a entrada consiste dos valores de m e n (número de linhas e de colunas da matriz) seguidos de triplas *(i, j, valor)* para os elementos diferentes de zero da matriz. Por

exemplo, para a matriz acima, a entrada seria:

```
4,  4
1,  1,    50.0
2,  1,    10.0
2,  3,    20.0
4,  1,   -30.0
4,  3,   -60.0
4,  4,     5.0
```

b) procedure ApagaMatriz (var A: Matriz);
Este procedimento devolve todas as células da matriz A para a área de memória disponível (use a **procedure** *dispose).*

c) procedure SomaMatriz (var A, B, C: Matriz);
Este procedimento recebe como parâmetros as matrizes A e B, devolvendo em C a soma de *A* com *B*.

d) procedure MultiplicaMatriz (var A, B, C: Matriz);
Este procedimento recebe como parâmetros as matrizes A e B, devolvendo em C o produto de *A* por *B*.

e) procedure ImprimeMatriz (var A: Matriz);
Este procedimento imprime (uma linha da matriz por linha da saída) a matriz A, *inclusive os* elementos iguais a zero.

Para inserir e retirar células das listas que formam a matriz, crie procedimentos especiais para este fim. Por exemplo, um procedimento

procedure Insere (i, j: integer; v: real; var A: Matriz);

para inserir o valor v na linha *i,* coluna *j* da matriz A será útil tanto na **procedure** LeMatriz quanto na **procedure SomaMatriz.**
As matrizes a serem lidas para testar os procedimentos são:

a) A mesma da Figura 2.10 deste enunciado;

b) $\begin{pmatrix} 50 & 30 & 0 & 0 \\ 10 & 0 & -20 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -5 \end{pmatrix}$

c) $\begin{pmatrix} 3 & 0 & 0 \\ 0 & -1 & 0 \end{pmatrix}$

Os procedimentos deverão ser testados com o seguinte programa:

```
program TestaMatrizesEsparsas;
...
...
begin
    ...
    LeMatriz (A); ImprimeMatriz (A);
    LeMatriz (B); ImprimeMatriz (B);
    SomaMatriz (A, B, C); ImprimeMatriz (C); ApagaMatriz (C);
    MultiplicaMatriz (A, B, C); ImprimeMatriz (C);
    ApagaMatriz (B); ApagaMatriz (C);
    LeMatriz (B);
    ImprimeMatriz (A); ImprimeMatriz (B);
    SomaMatriz (A, B, C); ImprimeMatriz (C);
    MultiplicaMatriz (A, B, C); ImprimeMatriz (C);
    MultiplicaMatriz (B, B, C);
    ImprimeMatriz (B); ImprimeMatriz (B); ImprimeMatriz (C);
    ApagaMatriz (A); ApagaMatriz (B); ApagaMatriz (C);
    ...
end. { TestaMatrizesEsparsas }
```

O que deve ser apresentado:

a) Listagem do programa em Pascal.

b) Listagem dos testes executados.

c) Descrição sucinta (por exemplo, desenho), das estruturas de da-dos e as decisões tomadas relativas aos casos e detalhes de especificação que porventura estejam omissos no enunciado.

d) Estudo da complexidade do tempo de execução dos procedimentos implementados e do programa como um todo (notação 0).

É obrigatório o uso de alocação dinâmica de memória para implementar as listas de adjacência que representam as matrizes. A análise de complexidade deve ser feita em função de *m, n* (dimensões da matriz) e r (número de elementos diferente de zero).

4) Considere F uma fila não vazia e P uma pilha vazia. Usando apenas a variável temporária x, e as quatro operações

$$x \Leftarrow P\ ,\ P \Leftarrow x\ ,\ x \Leftarrow F\ ,\ F \Leftarrow x$$

e os dois testes P=vazio e F=vazio, escreva um algoritmo para reverter a ordem dos elementos em F.

5) Duas pilhas podem coexistir em um mesmo vetor, uma **crescendo** em um sentido, e a outra no outro. Duas filas, ou uma pilha e uma fila podem também ser alocadas no mesmo vetor com o mesmo grau de eficiência ? Por quê ?

6) Como resolver o problema da representação por alocação seqüencial (usando uma região fixa de memória) para mais de duas pilhas ?

7) Alteração do ET

Alterar a especificação do programa ET, apresentado na seção 2.2.2, para aceitar o comando cancela palavra, cujo efeito é cancelar a **palavra** anterior na linha que está sendo editada. Por exemplo, se `$' é o CancelaPalavra, então a seqüência de caracteres NÃO CANCELA$$ CANCELA LINHA$ PALAVRA corresponde à seqüência CANCELA PALAVRA.

Altere o programa de acordo com a nova especificação proposta. Para testar o programa utilize o texto da página 53, com as seguintes alterações:

a) insira o caractere $ após a palavra "extras" na terceira linha,

b) insira dois caracteres $$ após as palavras "Será que" na penúltima linha do texto.

8) Existem partes de sistemas operacionais que cuidam da ordem em que os programas devem ser executados. Por exemplo, em um sistema de computação de tempo-compartilhado ("time-shared") existe a necessidade de manter um conjunto de processos em uma fila, esperando para serem executados.

Escreva um programa em Pascal que seja capaz de ler uma série de solicitações para: (i) incluir novos processos na fila de processos; (ii) retirar da fila o processo com o maior tempo de espera; (iii) imprimir o conteúdo da lista de processos em um determinado momento. Assuma que cada processo é representado por um registro composto por um número identificador do processo. Utilize o tipo abstrato de dados Fila apresentado na Seção 2.3.

9) Se você tem que escolher entre uma representação por lista **encadeada** ou uma representação usando posições contíguas de memória para um vetor, quais informações são necessárias para você selecionar uma representação apropriada? Como estes fatores influenciam a escolha da representação?

10) Filas, Simulação (Árabe, 1992)

O objetivo deste exercício é simular os padrões de aterrissagem e decolagem em um aeroporto. Suponha um aeroporto que possui 3 pistas, numeradas 1, 2 e 3. Existem 4 (quatro) "prateleiras" de espera para aterrissagem, duas para cada uma das pistas 1 e 2. Aeronaves que se aproximam do aeroporto devem se integrar a uma das prateleiras (filas) de espera, sendo que estas filas devem procurar manter o mesmo tamanho. Assim que um avião entra em uma fila de aterrissagem, ele recebe um número de identificação ID e um outro número inteiro que indica o número de unidades de tempo que o avião pode permanecer na fila antes que ele tenha que descer (do contrário seu combustível termina e ele cai).

Existem também filas para decolagem, uma para cada pista. Os aviões que chegam nestas filas também recebem uma identificação ID. Estas filas também devem procurar manter o mesmo tamanho.

A cada unidade de tempo, de 0 a 3 aeronaves podem chegar nas filas de decolagem e de 0 a 3 aeronaves podem chegar nas prateleiras. A cada unidade de tempo, cada pista pode ser usada para um pouso ou uma decolagem. A pista 3 em geral só é usada para decolagens, a não ser que um dos aviões nas prateleiras fique sem combustível, quando então ela deve ser imediatamente usada para pouso. Se apenas uma aeronave está com falta de combustível, ela pousará na pista 3; se mais de um avião estiver nesta situação, as outras pistas poderão ser utilizadas (a cada unidade de tempo no máximo 3 aviões poderão estar nesta desagradável situação).

Utilize inteiros pares (ímpares) sucessivos para a ID dos aviões chegando nas filas de decolagem (aterrissagem). A cada unidade de tempo, assuma que os aviões entram nas filas antes que aterrissagem ou decolagens ocorram. Tente projetar um algoritmo que não permita o crescimento excessivo das filas de aterrissagem ou decolagem. Coloque os aviões sempre no final das filas, que não devem ser reordenadas.

A saída do programa deverá indicar o que ocorre a cada unidade de tempo. Periodicamente imprima:

a) o conteúdo de cada fila;
b) o tempo médio de espera para decolagem;
c) o tempo médio de espera para aterrissagem; e
d) o número de aviões que aterrissam sem reserva de combustível.

Os itens b e c acima devem ser calculados para os aviões que já decolaram ou pousaram, respectivamente. A saída do programa deve ser auto-explicativa e fácil de entender.

A entrada poderia ser criada manualmente, mas o melhor é utilizar um gerador de números aleatórios. Para cada unidade de tempo, a entrada deve ter as seguintes informações:

a) número de aviões (0-3) chegando nas filas de aterrissagem com respectivas reservas de combustível (de 1 a 20 em unidades de tempo);
b) número de aviões (0-3) chegando nas filas de decolagem.

0 que deve ser apresentado:

a) Listagem dos programas em Pascal.
b) Listagem dos testes executados.
c) Descrição sucinta (por exemplo, desenho), das estruturas de da-dos e as decisões tomadas relativas aos casos e detalhes de especificação que porventura estejam omissos no enunciado.
d) Estudo da complexidade do tempo de execução dos procedimentos implementados e dos programas como um todo (notação O).